

PHP

- ❖ Write a PHP script to demonstrate variables in PHP. Use Globals, local & global keyword to access variables

```
<?php
// Global variable
$globalVar = "I am a global variable";
function demoVariables() {
// Local variable
    $localVar = "I am a local variable";
// Accessing global variable inside the function using global keyword
    global $globalVar;

    echo "Inside the function:<br>";
    echo "Local variable: $localVar <br>";
    echo "Global variable: $globalVar <br>";
}
// Accessing global variable outside the function
echo "Outside the function:<br>";
echo "Global variable: $globalVar <br>";
demoVariables();
?>
```

- ❖ Write a PHP script to demonstrate any 4 operators

```
<?php
// Arithmetic operators
$num1 = 10;
$num2 = 5;

$addition = $num1 + $num2;
$subtraction = $num1 - $num2;
$multiplication = $num1 * $num2;
$division = $num1 / $num2;

echo "Arithmetic Operators:<br>";
echo "Addition: $addition <br>";
echo "Subtraction: $subtraction <br>";
echo "Multiplication: $multiplication <br>";
echo "Division: $division <br>";

// Assignment operators
$x = 10;
$y = 5;

$x += $y; // Equivalent to: $x = $x + $y;
echo "Assignment Operators:<br>";
echo "x += y: $x <br>";

// Comparison operators
$a = 10;
$b = 5;
$c = 10;
```

```

echo "Comparison Operators:<br>";
echo "a == b: ";
var_dump($a == $b); // Output: false
echo "<br>";

echo "a == c: ";
var_dump($a == $c); // Output: true
echo "<br>";

```

// Logical operators

```

$p = true;
$q = false;

echo "Logical Operators:<br>";
echo "p AND q: ";
var_dump($p && $q); // Output: false
echo "<br>";

echo "p OR q: ";
var_dump($p || $q); // Output: true
echo "<br>";

```

?>

❖ What is Var- dump() function in PHP ? with example

In PHP, var_dump() is a function used for displaying structured information (type and value) about one or more variables. It's commonly used for debugging purposes to inspect the contents of variables.

Here's an example demonstrating the usage of var_dump():

<?php

```

// Example variables
$string = "Hello, world!";
$integer = 42;
$float = 3.14;
$array = [1, 2, 3];
$associativeArray = ["name" => "John", "age" => 30];
$object = new stdClass();
$object->property = "value";

// Displaying information about the variables
echo "Information about the variables using var_dump():<br>";
var_dump($string);
var_dump($integer);
var_dump($float);
var_dump($array);
var_dump($associativeArray);
var_dump($object);

```

?>

OUTPUT :

Information about the variables using var_dump():
string(13) "Hello, world!"

```

int(42)
float(3.14)

array(3) {
    [0]=> int(1)
    [1]=> int(2)
    [2]=> int(3)
}

array(2) {
    ["name"]=> string(4) "John"
    ["age"]=> int(30)
}

object(stdClass)#1 (1) {
    ["property"]=> string(5) "value"
}

```

As you can see, **var_dump()** provides detailed information about each variable, including its type and value. For strings, integers, floats, and arrays, it shows the length or count of elements as well. For objects, it shows the class name and properties.

❖ **Write a PHP script to display employees belonging to IT department and salary is in between 30,000 – 80,000 and store found records into another table.** (Assume suitable table structure)

Sure, let's break down the task into steps:

- Connect to the database.
- Retrieve records of employees belonging to the IT department and having a salary between 30,000 and 80,000.
- Store the found records into another table.

<?php

```

// Step 1: Connect to the database
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "your_database";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Step 2: Retrieve records of employees belonging to the IT department and having a salary between 30,000 and 80,000
$sql = "SELECT * FROM employees WHERE department = 'IT' AND salary BETWEEN 30000 AND 80000";
$result = $conn->query($sql);

// Step 3: Store the found records into another table
$newTableName = "filtered_employees"; // Replace with your desired table name
// Create a new table for storing filtered employees
$createTableSql = "CREATE TABLE IF NOT EXISTS $newTableName (

```

```

        id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        department VARCHAR(50) NOT NULL,
        salary DECIMAL(10,2) NOT NULL
    );
    $conn->query($createTableSql);

```

// Insert filtered records into the new table

```

    if($result->num_rows > 0) {
        while($row = $result->fetch_assoc()) {
            $name = $row["name"];
            $department = $row["department"];
            $salary = $row["salary"];

```

```

            $insertSql = "INSERT INTO $newTableName (name, department, salary) VALUES ('$name',
            '$department', '$salary')";

```

```

            $conn->query($insertSql);

```

```

        }

```

```

        echo "Records inserted successfully into $newTableName";

```

```

    }

```

```

    else

```

```

    {

```

```

        echo "No records found.";

```

```

    }

```

// Close connection

```

    $conn->close();

```

?>

- ❖ Write a PHP script & insert at least 5 records into it & update specific record in database. Assume student table with required fields in database.

<?php

// Step 1: Connect to the database

```

    $servername = "localhost";

```

```

    $username = "your_username";

```

```

    $password = "your_password";

```

```

    $dbname = "your_database";

```

// Create connection

```

    $conn = new mysqli($servername, $username, $password, $dbname);

```

// Check connection

```

    if($conn->connect_error) {

```

```

        die("Connection failed: " . $conn->connect_error);

```

```

    }

```

// Step 2: Insert at least 5 records into the student table

```

    $sql_insert = "INSERT INTO student (name, age, grade) VALUES

```

```

        ('John', 20, 'A'),

```

```

        ('Alice', 21, 'B'),

```

```

        ('Bob', 19, 'A'),

```

```

        ('Emma', 22, 'B'),

```

```

('Michael', 20, 'C');"

if ($conn->query($sql_insert) === TRUE) {
    echo "Records inserted successfully<br>";
}
else
{
    echo "Error inserting records: " . $conn->error;
}

```

// Step 3: Update a specific record in the student table

```

$student_id_to_update = 2;      // Assuming you want to update the record with id 2
$new_grade = 'A+';

$sql_update = "UPDATE student SET grade='$new_grade' WHERE id=$student_id_to_update";

if ($conn->query($sql_update) === TRUE) {
    echo "Record updated successfully<br>";
}
else
{
    echo "Error updating record: " . $conn->error;
}

// Close connection
$conn->close();
?>

```

❖ Write a PHP script to create a database using MYSQL.

<?php

```

$servername = "localhost";
$username = "your_username";
$password = "your_password";

// Create connection
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Create database
$dbname = "my_database";      // Replace with your desired database name

$sql = "CREATE DATABASE $dbname";

if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
}
else
{
    echo "Error creating database: " . $conn->error;
}

```

```
// Close connection
$conn->close();

?>
```

❖ What is Define function in PHP ? with example

In PHP, the **define()** function is used to define a constant. Constants are identifiers (names) for simple values that do not change during the execution of the script. By convention, constant identifiers are always uppercase.

Here's the syntax of the **define()** function:

```
define(name, value, case_insensitive);
```

- **name:** The name of the constant.
- **value:** The value of the constant.
- **case_insensitive:** Optional. If set to **true**, the constant name will be case-insensitive. Default is **false**.

Here's an example demonstrating the usage of the **define()** function:

```
<?php

// Define a constant
define("SITE_NAME", "My Website");

// Usage of constant
echo SITE_NAME;           // Output: My Website

?>
```

In this example, we define a constant named **SITE_NAME** with the value **"My Website"**. Once defined, we can use this constant throughout the script, and its value cannot be changed during the execution of the script. Constants are useful for storing values that are used repeatedly throughout your application and should not be changed.

❖ Write a PHP script to design a form for exam registration. Insert 5 records in database and display all the inserted records on new page.

Code:-

exam_registration_form.php:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exam Registration Form</title>
  </head>
  <body>
    <h2>Exam Registration Form</h2>
    <form action="insert_records.php" method="post">
```

Aniket_mh29

```

        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name" required><br>

        <label for="email">Email:</label><br>
        <input type="email" id="email" name="email" required><br>

        <label for="exam">Exam:</label><br>
        <select id="exam" name="exam" required>
            <option value="Math">Math</option>
            <option value="Science">Science</option>
            <option value="English">English</option>
        </select><br><br>

        <input type="submit" value="Register">

    </form>
</body>
</html>

```

insert_records.php:

```

<?php
    // Connect to the database
    $servername = "localhost";
    $username = "your_username";
    $password = "your_password";
    $dbname = "your_database";

    $conn = new mysqli($servername, $username, $password, $dbname);

    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    // Insert 5 records into the database
    for ($i = 1; $i <= 5; $i++) {
        $name = $_POST['name'] . $i;
        $email = $_POST['email'] . $i;
        $exam = $_POST['exam'];
        $sql = "INSERT INTO exam_registration (name, email, exam) VALUES ('$name', '$email', '$exam')";
        $conn->query($sql);
    }

    // Close connection
    $conn->close();

    // Redirect to the page to display inserted records
    header("Location: display_records.php");
    exit();
?>

```

display_records.php:

```

<?php
    // Connect to the database
    $servername = "localhost";

```

```

$username = "your_username"; // Replace with your MySQL username
$password = "your_password"; // Replace with your MySQL password
$dbname = "your_database"; // Replace with your database name

$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Fetch all records from the database
$sql = "SELECT * FROM exam_registration";
$result = $conn->query($sql);

// Display all inserted records
if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "Name: " . $row["name"]. " - Email: " . $row["email"]. " - Exam: " . $row["exam"]. "<br>";
    }
}
else
{
    echo "No records found.";
}

// Close connection
$conn->close();
?>

```

-
- ❖ Write a PHP script to design a form to reschedule the journey dates of flights and display the updated information on new web page.

Code:-

reschedule_form.php:

```

<!DOCTYPE html>
<html>
    <head>
        <title>Reschedule Flight Journey</title>
    </head>
    <body>
        <h2>Reschedule Flight Journey</h2>
        <form action="update_flight.php" method="post">
            <label for="flight_number">Flight Number:</label><br>
            <input type="text" id="flight_number" name="flight_number" required><br>
            <label for="new_departure_date">New Departure Date:</label><br>
            <input type="date" id="new_departure_date" name="new_departure_date" required><br>
            <label for="new_arrival_date">New Arrival Date:</label><br>
            <input type="date" id="new_arrival_date" name="new_arrival_date" required><br><br>
            <input type="submit" value="Reschedule">
        </form>
    </body>
</html>

```

```
</body>
</html>
```

update_flight.php:

```
<?php
    // Connect to the database
    $servername = "localhost";
    $username = "your_username"; // Replace with your MySQL username
    $password = "your_password"; // Replace with your MySQL password
    $dbname = "your_database"; // Replace with your database name

    $conn = new mysqli($servername, $username, $password, $dbname);

    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    // Get form data
    $flight_number = $_POST['flight_number'];
    $new_departure_date = $_POST['new_departure_date'];
    $new_arrival_date = $_POST['new_arrival_date'];

    // Update flight journey dates
    $sql = "UPDATE flights SET departure_date='$new_departure_date',
    arrival_date='$new_arrival_date' WHERE flight_number='$flight_number'";

    if ($conn->query($sql) === TRUE) {
        echo "Flight journey successfully rescheduled.<br>";
    }
    else
    {
        echo "Error updating flight: " . $conn->error;
    }

    // Close connection
    $conn->close();
?>
```

display_updated_flight.php:

```
<?php
    // Connect to the database
    $servername = "localhost";
    $username = "your_username"; // Replace with your MySQL username
    $password = "your_password"; // Replace with your MySQL password
    $dbname = "your_database"; // Replace with your database name

    $conn = new mysqli($servername, $username, $password, $dbname);

    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    // Fetch updated flight information
    $flight_number = $_POST['flight_number'];
```

```
$sql = "SELECT * FROM flights WHERE flight_number='$flight_number'";
$result = $conn->query($sql);
```

```
// Display updated flight information
```

```
if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "Flight Number: " . $row["flight_number"]. "<br>";
        echo "Departure Date: " . $row["departure_date"]. "<br>";
        echo "Arrival Date: " . $row["arrival_date"]. "<br>";
    }
}
else {
    echo "Flight not found.";
}
```

```
// Close connection
```

```
$conn->close();
```

```
?>
```

❖ What is an associative array in PHP? Explain with example.

In PHP, an associative array is a type of array where each element is associated with a specific key or name. Unlike indexed arrays, where elements are accessed by numeric indices, associative arrays allow you to access elements using their keys, which can be strings or integers.

Here's an example to illustrate associative arrays:

```
<?php
```

```
// Define an associative array
```

```
$student = array(
    "name" => "John",
    "age" => 20,
    "grade" => "A"
);
```

```
// Access elements of the associative array
```

```
echo "Name: " . $student["name"] . "<br>";
echo "Age: " . $student["age"] . "<br>";
echo "Grade: " . $student["grade"] . "<br>";
```

```
// Output: Name: John
```

```
// Output: Age: 20
```

```
// Output: Grade: A
```

```
// Modify an element
```

```
$student["age"] = 21;
```

```
// Add a new element
```

```
$student["city"] = "New York";
```

```
// Display the updated array
```

```
foreach ($student as $key => $value) {
    echo "$key: $value <br>";
}
```

```
?>
```

❖ Explain all array sorting methods in PHP with suitable example.

In PHP, there are various array sorting functions available to sort arrays in different ways. Here are the most commonly used array sorting methods in PHP along with suitable examples:

1. **sort()**: Sorts an array in ascending order.

```
<?php
    $numbers = array(4, 2, 8, 6, 3);
    sort($numbers);
    print_r($numbers); // Output: Array ( [0] => 2 [1] => 3 [2] => 4 [3] => 6 [4] => 8 )
?>
```

2. **rsort()**: Sorts an array in descending order.

```
<?php
    $numbers = array(4, 2, 8, 6, 3);
    rsort($numbers);
    print_r($numbers); // Output: Array ( [0] => 8 [1] => 6 [2] => 4 [3] => 3 [4] => 2 )
?>
```

3. **asort()**: Sorts an associative array in ascending order, according to the value.

```
<?php
    $age = array("Peter" => 30, "John" => 20, "Mary" => 25);
    asort($age);
    print_r($age); // Output: Array ( [John] => 20 [Mary] => 25 [Peter] => 30 )
?>
```

4. **arsort()**: Sorts an associative array in descending order, according to the value.

```
<?php
    $age = array("Peter" => 30, "John" => 20, "Mary" => 25);
    arsort($age);
    print_r($age); // Output: Array ( [Peter] => 30 [Mary] => 25 [John] => 20 )
?>
```

5. **ksort()**: Sorts an associative array in ascending order, according to the key.

```
<?php
    $age = array("Peter" => 30, "John" => 20, "Mary" => 25);
    ksort($age);
    print_r($age); // Output: Array ( [John] => 20 [Mary] => 25 [Peter] => 30 )
?>
```

6. **krsort()**: Sorts an associative array in descending order, according to the key.

```
<?php
    $age = array("Peter" => 30, "John" => 20, "Mary" => 25);
    krsort($age);
    print_r($age); // Output: Array ( [Peter] => 30 [Mary] => 25 [John] => 20 )
?>
```

7. **usort()**: Sorts an array using a user-defined comparison function.

```
<?php
    function custom_sort($a, $b) {
        return $a['age'] - $b['age'];
    }
```

```

    }
    $people = array(
        array('name' => 'John', 'age' => 20),
        array('name' => 'Mary', 'age' => 25),
        array('name' => 'Peter', 'age' => 30)
    );

    usort($people, 'custom_sort');
    print_r($people);
?>

```

❖ Explain static, local and global variables in PHP with suitable example.

Static Variables:

- Static variables are declared within a function and retain their value between function calls.
- They are initialized only once during the first function call and preserve their value for subsequent calls.
- Static variables are declared using the **static** keyword inside a function.

Example:

```

<?php
function increment() {
    static $count = 0;    // Static variable
    $count++;
    echo "Count: $count <br>";
}

increment();    // Output: Count: 1
increment();    // Output: Count: 2
increment();    // Output: Count: 3
?>

```

Local Variables:

- Local variables are declared within a function and are accessible only within that function.
- They have a limited scope and are destroyed once the function execution completes.

Example:

```

<?php
function greet() {
    $message = "Hello, world!"; // Local variable
    echo $message;
}

greet();    // Output: Hello, world!
echo $message;    // This will generate an error as $message is not accessible outside the function
?>

```

Global Variables:

- Global variables are declared outside of any function and can be accessed from any part of the script, including functions.
- They have a global scope and remain in memory throughout the execution of the script.

- To access global variables from within a function, the **global** keyword is used.

Example:

<?php

```
$globalVar = "I am a global variable";  
function displayGlobal() {  
    global $globalVar;           // Accessing global variable inside the function using the global keyword  
    echo "Global variable inside function: $globalVar <br>";  
}  
echo "Global variable outside function: $globalVar <br>";  
displayGlobal();               // Output: Global variable inside function: I am a global variable
```

?>

Node.Js

Node.js is an open-source, server-side JavaScript runtime environment that allows you to execute JavaScript code outside of a web browser. It's commonly used for building scalable network applications and is particularly popular for building web servers. With its event-driven, non-blocking I/O model, Node.js is efficient and lightweight, making it suitable for real-time applications.

Write a program in node js using event handling to perform all basic arithmetic operation

Code :-

```
const EventEmitter = require('events');  
  
class ArithmeticOperations extends EventEmitter  
{  
    add(a, b) {  
        this.emit('result', a + b);  
    }  
    subtract(a, b) {  
        this.emit('result', a - b);  
    }  
    multiply(a, b) {  
        this.emit('result', a * b);  
    }  
    divide(a, b) {  
        if (b === 0)  
        {  
            this.emit('error', 'Division by zero');  
        }  
    }  
}
```

```

        else {
            this.emit('result', a / b);
        }
    }
}

const calculator = new ArithmeticOperations();

calculator.on('result', (result) => {
    console.log('Result:', result);
});

calculator.on('error', (error) => {
    console.error('Error:', error);
});

calculator.add(5, 3);
calculator.subtract(10, 4);
calculator.multiply(2, 6);
calculator.divide(8, 2);
calculator.divide(5, 0);

```

Write a program in node js to perform file CURD operation by using URL module

Code :-

```

const http = require('http');
const fs = require('fs');
const url = require('url');

const server = http.createServer((req, res) => {
    const parsedUrl = url.parse(req.url, true);
    const path = parsedUrl.pathname;
    const query = parsedUrl.query;

    switch (req.method) {
        case 'GET':
            if (path === '/read') {
                fs.readFile('data.txt', 'utf8', (err, data) => {
                    if (err) {
                        res.writeHead(404);
                        res.end('File not found');
                    }
                });
            }
            else {
                res.writeHead(200, { 'Content-Type': 'text/plain' });
                res.end(data);
            }
        }
    });

    break;

    case 'POST':
        if (path === '/create') {
            const newData = query.data;

```

```

    fs.writeFile('data.txt', newData, (err) => {
      if (err) {
        res.writeHead(500);
        res.end('Error writing to file');
      } else {
        res.writeHead(201);
        res.end('Data created successfully');
      }
    });
  }
  break;
  case 'PUT':
    if (path === '/update') {
      const newData = query.data;
      fs.appendFile('data.txt', newData, (err) => {
        if (err) {
          res.writeHead(500);
          res.end('Error updating file');
        }
        else {
          res.writeHead(200);
          res.end('Data updated successfully');
        }
      });
    }
    break;
    case 'DELETE':
      if (path === '/delete') {
        fs.unlink('data.txt', (err) => {
          if (err) {
            res.writeHead(500);
            res.end('Error deleting file');
          } else {
            res.writeHead(200);
            res.end('File deleted successfully');
          }
        });
      }
      break;
      default:
        res.writeHead(404);
        res.end('Invalid endpoint');
        break;
    }
  });

const PORT = process.env.PORT || 3000;

server.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});

```

Write a program to read query string URL property in node JS

Code :-

```
const url = require('url');
// Sample URL with query string
const urlString = 'http://example.com/path?name=John&age=30';
// Parse the URL
const parsedUrl = url.parse(urlString, true);
// Access query parameters
const queryParameters = parsedUrl.query;

// Print query parameters
console.log('Query Parameters:', queryParameters);
console.log('Name:', queryParameters.name);
console.log('Age:', queryParameters.age);
```

Write a program using npm which will convert entered string into lowercase and uppercase

Code:-

✓ **npm** install *readline-sync chalk*

```
const readlineSync = require('readline-sync');
const chalk = require('chalk');

// Function to convert string to lowercase
function toLowerCase(str) {
  return str.toLowerCase();
}

// Function to convert string to uppercase
function toUpperCase(str) {
  return str.toUpperCase();
}

// Main function
function main() {
  // Get input string from the user
  const inputString = readlineSync.question('Enter a string: ');

  // Convert string to lowercase and uppercase
  const lowerCaseString = toLowerCase(inputString);
  const upperCaseString = toUpperCase(inputString);

  // Display the results
  console.log(chalk.green('Lowercase:'), lowerCaseString);
  console.log(chalk.blue('Uppercase:'), upperCaseString);
}

// Call the main function
main();
```

Explain event handling in node JS with a suitable example

Code :-

```
const EventEmitter = require('events');
// Create an event emitter instance
const myEmitter = new EventEmitter();

// Register an event listener for the 'greet' event
myEmitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

// Emit the 'greet' event with a parameter
myEmitter.emit('greet', 'John');
```

Write a program for multiplication of two number using event handling in node JS call multiplication function as an event cost

Code :-

```
const EventEmitter = require('events');

class Calculator extends EventEmitter {
  multiply(a, b) {
    // Emit the 'multiply' event with the result of multiplication
    this.emit('multiply', a * b);
  }
}

// Create an instance of Calculator
const calculator = new Calculator();

// Register an event listener for the 'multiply' event
calculator.on('multiply', (result) => {
  console.log('The result of multiplication:', result);
});

// Call the multiply function as an event
calculator.multiply(5, 3);
```

Write a program to show current date and time using user define model in node js

Code :-

[dateTime.js](#)

```
// Define a user-defined module to get the current date and time
module.exports.getCurrentDateTime = () => {
  const currentDate = new Date();
  return currentDate.toLocaleString();
};
```

[app.js](#)

```
// Import the user-defined module
const dateTime = require('./dateTime');

// Get the current date and time using the user-defined module
const currentDateTime = dateTime.getCurrentDateTime();

// Display the current date and time
console.log('Current date and time:', currentDateTime);
```

- ✓ Run the program by executing `node app.js`

Angular

AngularJS is a JavaScript framework developed by Google for building dynamic web applications. It's popular for its two-way data binding and modular structure, which makes it easier to develop and maintain complex web applications. However, it's worth noting that AngularJS has been succeeded by Angular (also known as Angular 2+), which is a complete rewrite and offers more features and performance improvements.

Create an angular program which will demonstrate the uses of **ngfor** directive and component

TYPESCRIPT :

```
// list.component.ts

import { Component } from '@angular/core';

@Component({
  selector: 'app-list',
  templateUrl: './list.component.html',
  styleUrls: ['./list.component.css']
})
export class ListComponent {
  items: string[] = ['Item 1', 'Item 2', 'Item 3', 'Item 4'];
}
```

HTML:

```
<!-- list.component.html -->
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>
```

TYPESCRIPT :

```
// app.module.ts

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { ListComponent } from './list/list.component';

@NgModule({
  declarations: [
    AppComponent,
    ListComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

HTML:

```
<!-- app.component.html -->
<app-list></app-list>
```

Write a program to demonstrate ngif and ngswitch statement

TYPESCRIPT:

```
// app.component.ts
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  showSection1: boolean = true;
  selectedOption: number = 1;

  toggleSection(): void {
    this.showSection1 = !this.showSection1;
  }
}
```

HTML:

```
<!-- app.component.html -->

<button (click)="toggleSection()">Toggle Section</button>

<div *ngIf="showSection1">
  <h2>Section 1</h2>
  <p>This is the content of section 1.</p>
</div>

<div *ngIf="!showSection1">
  <h2>Section 2</h2>
  <p>This is the content of section 2.</p>
</div>
<hr>
<select [(ngModel)]="selectedOption">
  <option value="1">Option 1</option>
  <option value="2">Option 2</option>
  <option value="3">Option 3</option>
</select>

<div [ngSwitch]="selectedOption">
  <div *ngSwitchCase="1">
    <p>Option 1 is selected.</p>
  </div>
  <div *ngSwitchCase="2">
    <p>Option 2 is selected.</p>
  </div>
</div>
```

```

    <div *ngSwitchCase="3">
      <p>Option 3 is selected.</p>
    </div>
    <div *ngSwitchDefault>
      <p>No option selected.</p>
    </div>
  </div>

```

Create angular program which will demonstrate the uses of component directive

TYPESCRIPT :

```

// highlight.directive.ts
import { Directive, ElementRef, HostListener } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) { }

  @HostListener('mouseenter') onMouseEnter() {
    this.highlight('yellow');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.highlight(null);
  }

  private highlight(color: string | null) {
    this.el.nativeElement.style.backgroundColor = color;
  }
}

// app.component.ts

import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

}

```

HTML:

```

<!-- app.component.html -->

<div appHighlight>
  Hover over me to see the effect!
</div>

```

Write a program to display string in uppercase and lowercase using Angular JS filters

Code:-

```
<!-- index.html -->

<!DOCTYPE html>
<html lang="en" ng-app="myApp">
  <head>
    <meta charset="UTF-8">
    <title>AngularJS Filters Example</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  </head>
  <body>

    <div ng-controller="myCtrl">
      <h2>Uppercase and Lowercase Filters</h2>

      <p>Original String: {{ originalString }}</p>
      <p>Uppercase: {{ originalString | uppercase }}</p>
      <p>Lowercase: {{ originalString | lowercase }}</p>
    </div>

    <script>
      var app = angular.module('myApp', []);
      app.controller('myCtrl', function($scope) {
        $scope.originalString = 'Hello World!';
      });
    </script>

  </body>
</html>
```

Explain ng-app, ng-model and ng-bind with suitable example

1. **ng-app**:

- The **ng-app** directive is used to define the root element of an AngularJS application.
- It initializes the AngularJS framework and tells it which part of the HTML document it should manage.
- You typically place **ng-app** on the **<html>** or **<body>** tag to bootstrap the entire application, or on a specific element if you want to limit the scope of the AngularJS application.

Code:-

```
<!DOCTYPE html>

<html lang="en" ng-app="myApp">
  <head>
    <meta charset="UTF-8">
    <title>AngularJS Example</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
```

```
</head>

<body>

  <div ng-controller="myCtrl">

    <!-- Content managed by AngularJS -->

  </div>

</body>

</html>
```

2. ng-model:

- The **ng-model** directive binds the value of HTML controls (like input, select, textarea) to application data.
- It provides two-way data binding, meaning changes in the UI automatically update the model and vice versa.

Code:-

```
<div ng-app="myApp" ng-controller="myCtrl">

  <input type="text" ng-model="name">

  <p>Hello, {{ name }}</p>

</div>

<script>

  var app = angular.module('myApp', []);

  app.controller('myCtrl', function($scope) {

    $scope.name = 'John';

  });

</script>
```

3. ng-bind:

- The **ng-bind** directive is used to bind the content of an HTML element to an expression.
- It is similar to using double curly braces **{{ expression }}**, but it's more efficient, especially when dealing with large amounts of data.
- It prevents the display of the raw expression before AngularJS has a chance to parse it.

Code:-

```
<div ng-app="myApp" ng-controller="myCtrl">

  <p ng-bind="message"></p>

</div>

<script>

  var app = angular.module('myApp', []);

  app.controller('myCtrl', function($scope) {

    $scope.message = 'Hello, AngularJS!';

  });

</script>
```

Create a login from both username and password. It's mandatory after entering the value transfer user control to the next web page. Show the message as you have login successfully. Use ng-href, ng-required

Code:-

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
  <head>
    <meta charset="UTF-8">
    <title>Login Form</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  </head>
  <body>

    <div ng-controller="LoginController">
      <h2>Login</h2>
      <form ng-submit="login()">
        <div>
          <label>Username:</label>
          <input type="text" ng-model="username" ng-required="true">
        </div>

        <div>
          <label>Password:</label>
          <input type="password" ng-model="password" ng-required="true">
        </div>
        <button type="submit" ng-disabled="loginForm.$invalid">Login</button>
      </form>
      <p ng-if="loggedIn">You have logged in successfully!</p>
    </div>

    <script>
      var app = angular.module('myApp', []);
      app.controller('LoginController', function($scope, $location) {
        $scope.login = function() {
          // Check if username and password are provided
          if ($scope.username && $scope.password)
          {
            // Simulate login success (you would normally perform authentication here)
            // For demonstration, just setting loggedIn to true
            $scope.loggedIn = true;
            // Redirect to the next web page
            $location.path('/next-page');
          }
        };
      });
    </script>

  </body>
</html>
```

Explain Routing in Angular JS with a suitable example

1. First, include the AngularJS library and the AngularJS route module in your HTML file:

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
  <head>
    <meta charset="UTF-8">
    <title>AngularJS Routing Example</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-route.js"></script>
  </head>

  <body>
    <div ng-view></div>
    <script>
      var app = angular.module('myApp', ['ngRoute']);
    </script>

  </body>
</html>
```

2. Next, define your routes and their corresponding templates and controllers:

```
<script>
  app.config(function($routeProvider) {
    $routeProvider
      .when('/', {
        templateUrl: 'home.html',
        controller: 'HomeController'
      })
      .when('/about', {
        templateUrl: 'about.html',
        controller: 'AboutController'
      })
      .when('/contact', {
        templateUrl: 'contact.html',
        controller: 'ContactController'
      })
      .otherwise({
        redirectTo: '/'
      })
  });

  app.controller('HomeController', function($scope) {
    $scope.message = 'Welcome to the Home Page!';
  });

  app.controller('AboutController', function($scope) {
    $scope.message = 'This is the About Page!';
  });

  app.controller('ContactController', function($scope) {
    $scope.message = 'Contact us at: example@example.com!';
  });
</script>
```

3. Create HTML templates for each route:

Content.html

```
<h2>{{ message }}</h2>
<p>This is the home page content.</p>
```

About.html

```
<h2>{{ message }}</h2>
<p>This is the about page content.</p>
```

Contact.html

```
<h2>{{ message }}</h2>
<p>Contact us at: example@example.com</p>
```

HTML 5

Write a program using <Canvas> tag to draw different shapes (Circle, Triangle and Rectangle)

Code:-

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Canvas Shapes</title>
    <style>
      canvas {
        border: 1px solid black;
      }
    </style>
  </head>
  <body>
    <canvas id="myCanvas" width="500" height="500"> </canvas>
    <script>
      window.onload = function() {
        const canvas = document.getElementById('myCanvas');
        const ctx = canvas.getContext('2d');

        // Draw a circle
        ctx.beginPath();
        ctx.arc(150, 100, 50, 0, Math.PI * 2, false); // x, y, radius, startAngle, endAngle
        ctx.fillStyle = 'blue';
        ctx.fill();
        ctx.closePath();

        // Draw a rectangle
        ctx.beginPath();
        ctx.rect(50, 200, 200, 100); // x, y, width, height
        ctx.fillStyle = 'green';
        ctx.fill();
        ctx.closePath();

        // Draw a triangle
        ctx.beginPath();
        ctx.moveTo(300, 300); // Starting point
        ctx.lineTo(400, 300); // Line to the right
        ctx.lineTo(350, 200); // Line to the top
        ctx.lineTo(300, 300); // Back to the starting point
        ctx.fillStyle = 'red';
        ctx.fill();
        ctx.closePath();
      }
    </script>
  </body>
</html>
```

Write a program using <svg> tag to draw line, Rectangle & Triangle shapes

Code:-

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SVG Shapes</title>
    <style>
      svg {
        border: 1px solid black;
      }
    </style>
  </head>
  <body>
    <h2>SVG Shapes Example</h2>
    <svg width="500" height="500">
      <!-- Draw a line -->
      <line x1="50" y1="50" x2="450" y2="50" stroke="black" stroke-width="2" />

      <!-- Draw a rectangle -->
      <rect x="50" y="100" width="200" height="100" fill="blue" stroke="black" stroke-width="2" />

      <!-- Draw a triangle -->
      <polygon points="300,200 400,100 500,200" fill="green" stroke="black" stroke-width="2" />
    </svg>
  </body>
</html>
```

Explanation

1. Line:

- The `<line>` element is used to create a straight line.
- `x1` and `y1` are the coordinates for the starting point of the line.
- `x2` and `y2` are the coordinates for the ending point of the line.
- `stroke` defines the color of the line.
- `stroke-width` specifies the width of the line.

2. Rectangle:

- The `<rect>` element is used to draw a rectangle.
- `x` and `y` are the coordinates for the upper-left corner of the rectangle.
- `width` and `height` define the dimensions of the rectangle.
- `fill` specifies the fill color of the rectangle.
- `stroke` defines the color of the rectangle's border.
- `stroke-width` specifies the width of the border.

3. Triangle:

- The `<polygon>` element is used to create a shape with three or more sides (a polygon).

- **points** specifies the coordinates of the vertices of the polygon. In this example, it defines a triangle with vertices at (300,200), (400,100), and (500,200).
- **fill** specifies the fill color of the triangle.
- **stroke** defines the color of the triangle's border.
- **stroke-width** specifies the width of the border.

Explain Audio & Video tag in HTML with Example

<Audio> Tag :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Audio Example</title>
  </head>
  <body>
    <h2>Audio Example</h2>
    <audio controls>
      <source src="audio-file.mp3" type="audio/mpeg">
      <source src="audio-file.ogg" type="audio/ogg">
      Your browser does not support the audio element.
    </audio>
  </body>
</html>
```

<Video> Tag

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Video Example</title>
  </head>
  <body>
    <h2>Video Example</h2>
    <video width="640" height="360" controls>
      <source src="video-file.mp4" type="video/mp4">
      <source src="video-file.webm" type="video/webm">
      Your browser does not support the video tag.
    </video>
  </body>
</html>
```

Common Attributes and Methods

Both `<audio>` and `<video>` elements share several attributes and methods:

- **controls**: Adds playback controls.
- **autoplay**: The media will start playing as soon as it is ready.
- **loop**: The media will start over again, every time it is finished.
- **muted**: The audio output of the media will be muted.
- **preload**: Specifies if and how the author thinks that the media file should be loaded when the page loads. It can take the values **auto**, **metadata**, or **none**.

Example with Autoplay and Loop:

Code:-

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Media Example</title>
  </head>
  <body>
    <h2>Autoplay and Loop Audio Example</h2>
    <audio autoplay loop>
      <source src="audio-file.mp3" type="audio/mpeg">
      <source src="audio-file.ogg" type="audio/ogg">
      Your browser does not support the audio element.
    </audio>

    <h2>Autoplay and Loop Video Example</h2>
    <video width="640" height="360" autoplay loop muted>
      <source src="video-file.mp4" type="video/mp4">
      <source src="video-file.webm" type="video/webm">
      Your browser does not support the video tag.
    </video>
  </body>
</html>
```

